

Intern Assignment: YouTube Watch Party System

Overview

Build a **Watch Party** system that allows multiple users to watch YouTube videos together in real time. Users should be synchronized—when one person pauses, seeks, or changes the video, everyone in the party sees the same action.

Core Requirements

- Real-time synchronization** – All participants see the same video state (play/pause, seek position, current video)
- Room-based model** – Users create or join watch rooms with unique links/codes
- YouTube integration** – Play YouTube videos in sync for all room participants
- WebSockets** – Use WebSockets for real-time communication between server and clients
- Role-based access** – Rooms have roles; the host can assign roles to participants

Role-Based Access Control

Each room must support **roles** with different permissions. The **host** (creator) can assign roles to participants.

Roles

Role	Who assigns	Permissions
Host	Auto (room creator)	Full control: play/pause, seek, change video, assign roles, remove participants, transfer host
Moderator	Host	Play/pause, seek, change video; may optionally manage roles/participants if you choose
Participant	Host (default for joiners)	Watch only; cannot control playback or change video
Viewer	Host	Same as Participant (alias, if you want to distinguish)

You may simplify to Host + Participant only for MVP, but Host must be able to promote Participant Moderator (or similar).

Host Capabilities

- **Assign role** – Host selects a participant and assigns them a role (e.g., Participant Moderator)
- **Remove participant** – Host can remove a user from the room
- **Transfer host** – Host can pass the Host role to another participant (optional)

Role Enforcement

- Backend must validate permissions before processing events (e.g., reject `change_video` from a Participant)
- Broadcast role updates to the room so the UI can show roles and disable controls for restricted users

Tech Stack

Layer	Technology	Purpose
Frontend	React / Next.js or Vue	UI, room creation/join, video player
Backend	Node.js (Express) or Python (FastAPI)	API, room logic, WebSocket server
Real-time	Socket.IO or ws	WebSocket-based bidirectional communication
Database	PostgreSQL or MongoDB or SQLite	Rooms, user sessions, room metadata (optional for MVP)
Video	YouTube IFrame API	Embedded, controllable YouTube player

Recommended Stack (example)

- **Frontend:** React + TypeScript + Vite
- **Backend:** Node.js + Express or Next.js (full-stack)
- **WebSocket server:** Socket.IO or ws
- **Database:** SQLite / PostgreSQL (for rooms, optional)

Feel free to choose a stack you are familiar with as long as **WebSockets** are used for real-time communication.

WebSocket Events (Suggested)

Event	Direction	Payload	Description
-------	-----------	---------	-------------

join_room	Client Server	{ roomId, username }	User joins; server assigns role (Host if creator, else Participant)
leave_room	Client Server	{ roomId }	User leaves the room
sync_state	Server Clients	{ playState, currentTime, videoId }	Broadcast current video state to room
play	Client Server	{ }	User pressed play; requires Host/Moderator ; server broadcasts
pause	Client Server	{ }	User pressed pause; requires Host/Moderator ; server broadcasts
seek	Client Server	{ time }	User seeks; requires Host/Moderator ; server broadcasts
change_video	Client Server	{ videoId }	Change video; requires Host/Moderator ; server broadcasts
assign_role	Client Server	{ userId, role }	Host assigns role to participant; Host only
remove_participant	Client Server	{ userId }	Host removes user from room; Host only
user_joined	Server Clients	{ username, userId, role, participants }	New participant joined (participants include roles)
user_left	Server Clients	{ username, userId, participants }	Participant left
role_assigned	Server Clients	{ userId, username, role, participants }	Role was assigned; update participant list
participant_removed	Server Clients	{ userId, participants }	Participant was removed by host

Functional Requirements

- ☐ Create a new watch party room (creator becomes Host)
- ☐ Join an existing room via link or room code (joiner gets Participant by default)
- ☐ Display list of participants with their roles

- ☐ Host can assign roles to participants (e.g., Participant + Moderator)
- ☐ Host can remove participants from the room
- ☐ Playback controls (play/pause, seek, change video) restricted to Host and Moderator
- ☐ Play/pause synchronization
- ☐ Seek synchronization (when one user scrubs, others follow)
- ☐ Change video synchronization (e.g. paste new YouTube URL)
- ☐ Basic chat (optional bonus)

Deployment

Deploy your application so it is **publicly accessible** on the internet and fully working. Use one of these platforms:

Platform	Best for
Render	Full-stack apps, WebSocket servers, backend + frontend
Vercel	Frontend + serverless; works well with Next.js
Netlify	Frontend; use with a separate backend/WebSocket server
Railway	Full-stack, WebSocket servers, databases

- Your live app must be reachable via a public URL (e.g., `https://your-app.onrender.com`)
- Core features (create/join room, sync playback, roles) must work in production
- Include the live URL in your README

Code Understanding

You must be able to **explain what you have used** in your code. Be prepared to discuss:

- How each library/tool is used (e.g., Socket.IO, React, Express)
- How WebSockets enable real-time sync
- How the role-based logic works on the backend
- Any deployment choices (env vars, build steps, platform limits)
- Trade-offs or issues you ran into

Deliverables

1. **Working application** – Codebase that runs locally **and** deployed publicly (Render, Vercel, Netlify, or Railway)

2. **README.md** – Setup and run instructions + **live deployment URL**
 3. **Architecture overview** – Short description of how WebSockets integrate with the flow
 4. **Code walkthrough readiness** – Ability to explain the technologies and logic used
 5. **Demo** – Brief demo video or screenshots (optional)
-

Bonus Ideas

- **OOP concepts for WebSocket servers** – Structure your WebSocket server using OOP: e.g., `Room` class (participants, state, broadcast methods), `Participant` class, `MessageHandler` class, etc. Encapsulate room logic, validation, and event handling in well-defined classes.
 - **Scalability** – Design for many concurrent users and rooms: horizontal scaling with multiple WebSocket server instances, Redis Pub/Sub for cross-server broadcast, load balancer, connection pooling. Target 1,000+ users, 100+ rooms, 50+ users per room. See [Socket.IO Redis Adapter](#) and [Redis Pub/Sub](#).
 - Persistent rooms (save room ID/state in DB)
 - Authentication (login before joining)
 - Text chat in the room
 - Reactions / emoji reactions on key moments
 - Transfer host – Host can pass the Host role to another participant
-

Resources

- [YouTube IFrame Player API](#)
- [Socket.IO Docs](#)
- [WebSockets \(ws library\)](#)
- **Deployment:** [Render](#) | [Vercel](#) | [Netlify](#) | [Railway](#)